

NAME : DISHA SAMANTA

CUID : CU24250227

ROLLNO : 20

LABSHEET 10

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt

from tensorflow.keras import layers, models
from sklearn.metrics import confusion_matrix, classification_report
import seaborn as sns
```

```
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()

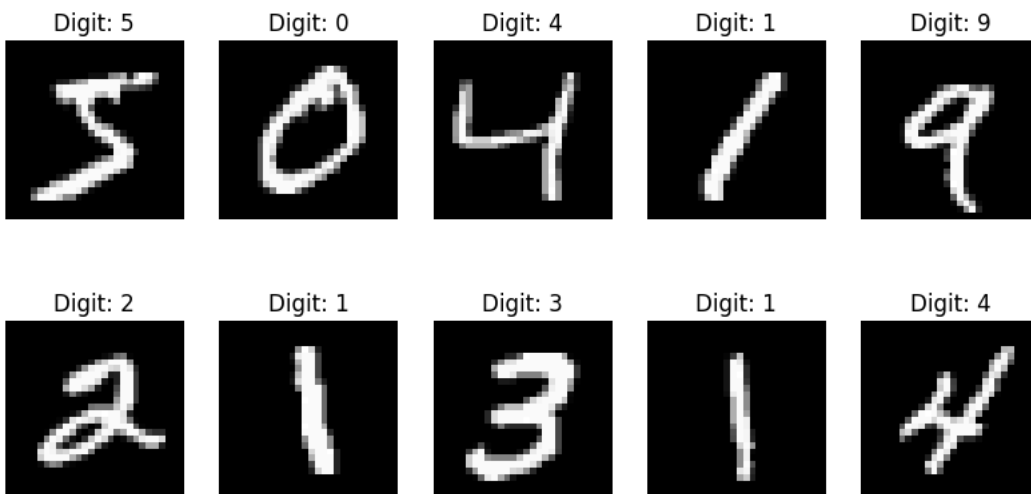
print("Training images:", x_train.shape)
print("Testing images:", x_test.shape)
```

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz>
11490434/11490434 ————— 2s 0us/step
Training images: (60000, 28, 28)
Testing images: (10000, 28, 28)

```
plt.figure(figsize=(10,5))

for i in range(10):
    plt.subplot(2,5,i+1)
    plt.imshow(x_train[i], cmap='gray')
    plt.title("Digit: " + str(y_train[i]))
    plt.axis('off')

plt.show()
```



```
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0

x_train = np.expand_dims(x_train, -1)
x_test = np.expand_dims(x_test, -1)

print("Training shape:", x_train.shape)
print("Testing shape:", x_test.shape)
```

Training shape: (60000, 28, 28, 1)
Testing shape: (10000, 28, 28, 1)

```
x_val = x_train[-5000:]
y_val = y_train[-5000:]

x_train_new = x_train[:-5000]
y_train_new = y_train[:-5000]

print("Training samples:", len(x_train_new))
print("Validation samples:", len(x_val))
print("Testing samples:", len(x_test))
```

Training samples: 55000
Validation samples: 5000
Testing samples: 10000

```
model = models.Sequential([

    layers.Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),
    layers.MaxPooling2D((2,2)),

    layers.Conv2D(64, (3,3), activation='relu'),
    layers.MaxPooling2D((2,2)),

    layers.Flatten(),

    layers.Dense(128, activation='relu'),
    layers.Dense(10, activation='softmax')
])

model.summary()
```

/usr/local/lib/python3.13/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to `Conv2D`/`Conv2DTranspose` layers. You can pass it to the first layer only.
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten (Flatten)	(None, 1600)	0
dense (Dense)	(None, 128)	204,928
dense_1 (Dense)	(None, 10)	1,290

Total params: 225,034 (879.04 KB)
Trainable params: 225,034 (879.04 KB)
Non-trainable params: 0 (0.00 B)

```
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
```

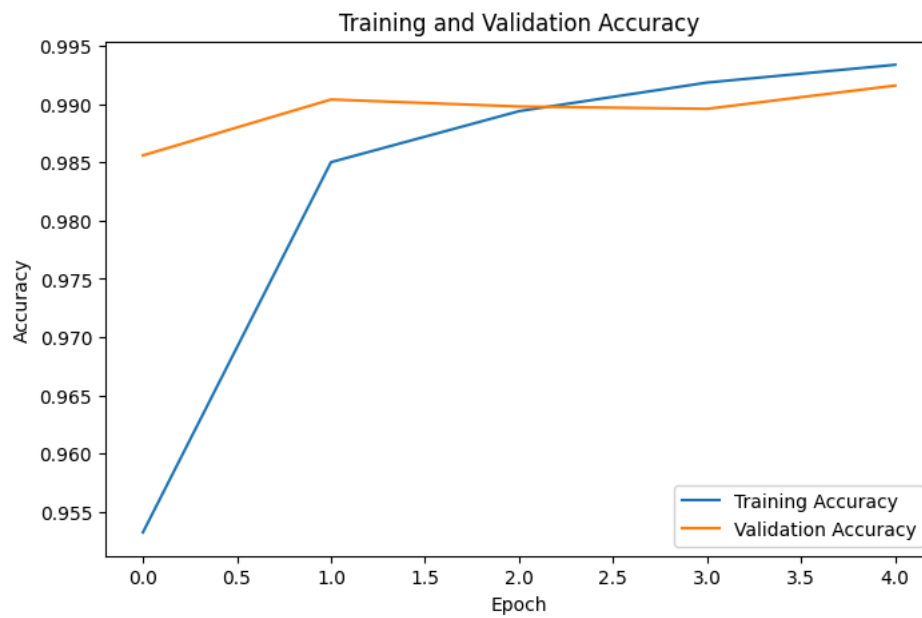
```
history = model.fit(
    x_train_new,
    y_train_new,
    epochs=5,
    batch_size=64,
    validation_data=(x_val, y_val)
)
```

Epoch 1/5
860/860 ————— 47s 54ms/step - accuracy: 0.9532 - loss: 0.1593 - val_accuracy: 0.9856 - val_loss: 0.0533
Epoch 2/5
860/860 ————— 80s 51ms/step - accuracy: 0.9850 - loss: 0.0502 - val_accuracy: 0.9904 - val_loss: 0.0394
Epoch 3/5
860/860 ————— 44s 51ms/step - accuracy: 0.9894 - loss: 0.0338 - val_accuracy: 0.9898 - val_loss: 0.0358
Epoch 4/5
860/860 ————— 43s 50ms/step - accuracy: 0.9919 - loss: 0.0252 - val_accuracy: 0.9896 - val_loss: 0.0368
Epoch 5/5
860/860 ————— 81s 49ms/step - accuracy: 0.9934 - loss: 0.0194 - val_accuracy: 0.9916 - val_loss: 0.0322

```
plt.figure(figsize=(8,5))

plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')

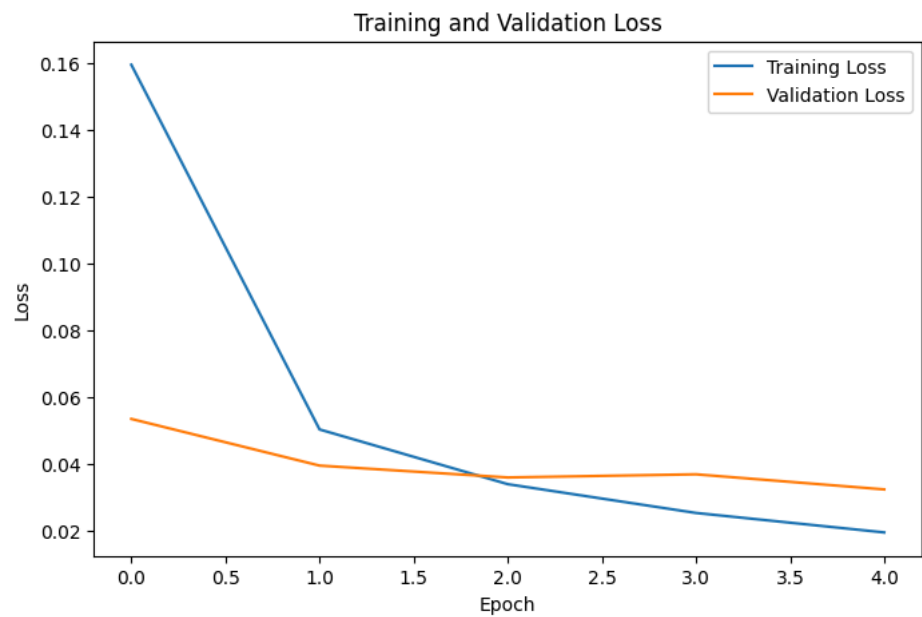
plt.title('Training and Validation Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.show()
```



```
plt.figure(figsize=(8,5))

plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')

plt.title('Training and Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.show()
```



```
test_loss, test_accuracy = model.evaluate(x_test, y_test, verbose=0)
```

```
print("Test Loss:", test_loss)
print("Test Accuracy:", test_accuracy)
```

```
Test Loss: 0.02602667547762394
Test Accuracy: 0.9915000200271606
```

```
predictions = model.predict(x_test)

predicted_labels = np.argmax(predictions, axis=1)

print("Actual Labels:")
print(y_test[:10])

print("\nPredicted Labels:")
print(predicted_labels[:10])
```

```
313/313 ————— 4s 12ms/step
```

```
Actual Labels:
[7 2 1 0 4 1 4 9 5 9]
```

```
Predicted Labels:
[7 2 1 0 4 1 4 9 5 9]
```

```

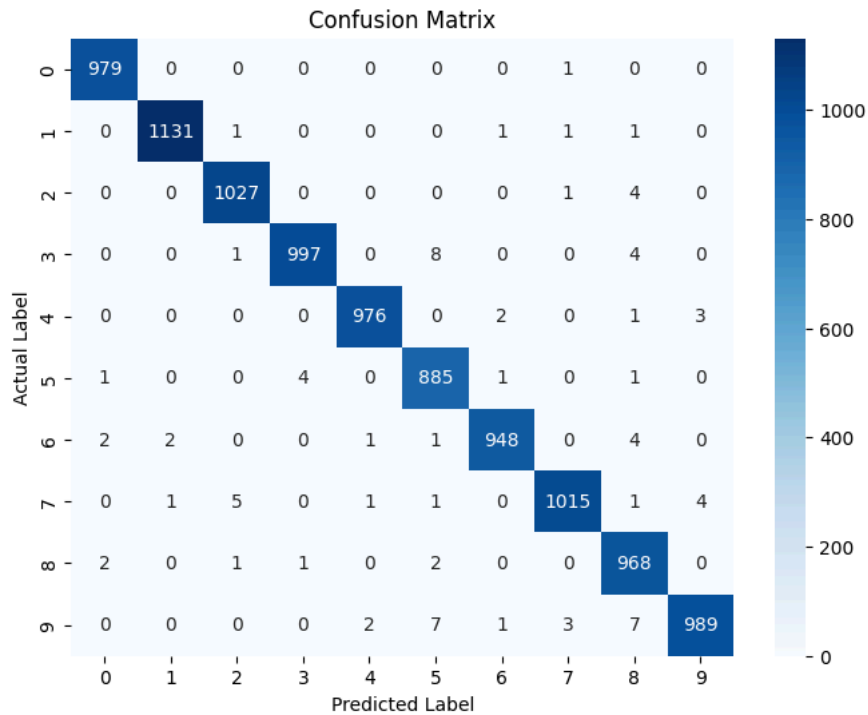
cm = confusion_matrix(y_test, predicted_labels)

plt.figure(figsize=(8,6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')

plt.title('Confusion Matrix')
plt.xlabel('Predicted Label')
plt.ylabel('Actual Label')

plt.show()

```



```
print(classification_report(y_test, predicted_labels))
```

	precision	recall	f1-score	support
0	0.99	1.00	1.00	980
1	1.00	1.00	1.00	1135
2	0.99	1.00	0.99	1032
3	1.00	0.99	0.99	1010
4	1.00	0.99	0.99	982
5	0.98	0.99	0.99	892
6	0.99	0.99	0.99	958
7	0.99	0.99	0.99	1028
8	0.98	0.99	0.99	974
9	0.99	0.98	0.99	1009
accuracy			0.99	10000
macro avg	0.99	0.99	0.99	10000
weighted avg	0.99	0.99	0.99	10000

```

index = 100

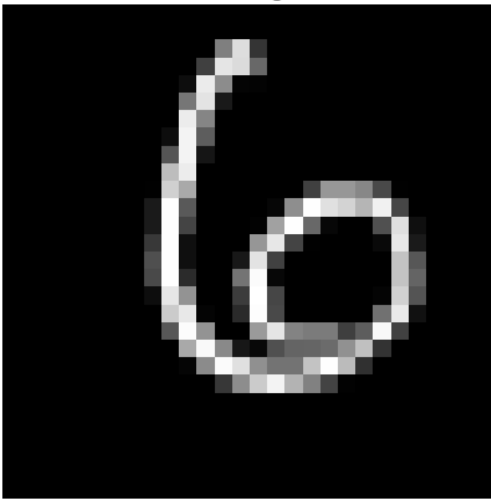
plt.imshow(x_test[index].reshape(28,28), cmap='gray')
plt.title("Actual Digit: " + str(y_test[index]))
plt.axis('off')
plt.show()

prediction = model.predict(x_test[index:index+1])
predicted_digit = np.argmax(prediction)

print("Actual Digit:", y_test[index])
print("Predicted Digit:", predicted_digit)

```

Actual Digit: 6



1/1 — 0s 47ms/step

Actual Digit: 6

Predicted Digit: 6

Questions and Answers

Q1. What is image classification? How does it differ from object detection and image segmentation?

Answer: Image classification is the process of assigning a class or category to an image. It tells what is present in the image but does not identify the exact location of the object. Object detection identifies and localizes one or more objects using bounding boxes along with their class labels. Image segmentation classifies individual pixels of an image and provides pixel-level information about different objects.

Q2. Explain the architecture and working principle of a Convolutional Neural Network (CNN).

Answer: A Convolutional Neural Network (CNN) is a deep learning model mainly used for image processing and classification. A typical CNN consists of convolutional layers, activation functions, pooling layers, flattening layers, fully connected layers, and an output layer.

The convolutional layers extract important features such as edges, shapes, and patterns from an image. Pooling layers reduce the spatial dimensions of the feature maps while retaining important information. The extracted features are then flattened and passed to fully connected layers for classification. Finally, the output layer predicts the class of the input image.

Q3. What is the role of Convolutional Layers, Pooling Layers, and Fully Connected Layers in a CNN?

Answer: Convolutional layers extract important features from an image using filters or kernels. They can detect features such as edges, textures, shapes, and patterns.

Pooling layers reduce the size of feature maps and decrease computational complexity while preserving important features. Max pooling is commonly used for this purpose.

Fully connected layers use the extracted features to perform the final classification. They connect the learned features to the output classes.

Q4. Why is image normalization performed before training a deep learning model?

Answer: Image normalization is performed to scale pixel values to a smaller and consistent range, usually between 0 and 1. For example, pixel values from 0 to 255 can be divided by 255. Normalization helps the neural network train faster and more efficiently. It also improves numerical stability and can help the model converge more effectively during training.

Q5. Explain the purpose of activation functions such as ReLU and Softmax in CNNs.

Answer: ReLU (Rectified Linear Unit) is commonly used in hidden layers of CNNs. It introduces non-linearity into the network and is defined as:

$$\text{ReLU}(x) = \max(0, x)$$

It helps the network learn complex patterns and features.

Softmax is generally used in the output layer for multi-class classification. It converts the output values into probabilities for each class, with the probabilities adding up to 1. The class with the highest probability is selected as the predicted class.

Q6. What is a Confusion Matrix? How is it used to evaluate classification performance?

Answer: A Confusion Matrix is a table used to compare the actual class labels with the predicted class labels of a classification model. It shows how many samples were correctly or incorrectly classified for each class.

For binary classification, it contains True Positive, True Negative, False Positive, and False Negative values. For multi-class classification such as MNIST, it shows the number of correct and incorrect predictions for each digit. It helps identify which classes are being confused by the model.

Q7. Differentiate between Accuracy, Precision, Recall, and F1-Score with suitable examples.

Answer: Accuracy is the percentage of total predictions that are correct.

$\text{Accuracy} = \text{Correct Predictions} / \text{Total Predictions}$

Precision measures how many of the samples predicted as a particular class are actually from that class.

$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$

Recall measures how many of the actual samples belonging to a class were correctly detected.

$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$

F1-Score is the harmonic mean of precision and recall. It provides a balance between precision and recall.

$\text{F1-Score} = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$

For example, if a CNN predicts handwritten digit 5, precision tells how many predicted 5s were actually 5, while recall tells how many actual 5s were correctly identified.

Q8. Compare traditional feature-based image classification methods such as SIFT/HOG + SVM with CNN-based classification.

Answer: Traditional methods such as SIFT and HOG require manual feature extraction before classification. These features are then given to a classifier such as SVM. CNNs automatically learn useful features directly from raw images during training.

Traditional feature-based methods are generally simpler and may require fewer computational resources, but their performance can depend heavily on the quality of manually selected features. CNNs can learn complex and hierarchical features and generally perform better on large image datasets, although they usually require more computational resources and training data.

Q9. Mention five real-world applications of CNN-based image classification.

Answer: Five real-world applications of CNN-based image classification are:

1. Handwritten digit and character recognition.
2. Face and facial expression recognition.
3. Medical image classification for disease detection.
4. Product and object recognition in retail systems.
5. Traffic sign and road scene classification in autonomous vehicles.

Q10. How can techniques such as Data Augmentation, Transfer Learning, and Hyperparameter Tuning improve the performance of deep learning-based image classification models?

Answer: Data Augmentation improves model performance by creating modified versions of training images using operations such as rotation, flipping, cropping, and scaling. It increases the variety of training data and can reduce overfitting.